

fiveup / ProofSignals Refactor Plan — Discord-backed matching, points and Skool tie-back

Sep 24, 2026 · @Robbie

The verdict

This is not a rewrite. About 70% of fiveup survives untouched: auth, profiles, assets, the feedback and star-rating core, moderation, the public profile layer, and the whole member graph (edges, separation degrees, match preferences). What changes is who triggers the writes. Today a member clicks Browse and the site creates a match; after the refactor a Squad, a Mission or an `/assist` in Discord creates the same rows. The web matching surface gets shelved, not deleted, behind one switch, so the day Discord changes its rules you flip it back.

Three things get added that do not exist anywhere in the repo yet: a points ledger with an admin approval queue, an accomplishments catalog, and Discord and Skool identity links. Two things get reused in ways they were not built for: `member_match_edges` becomes the relationship graph fed by Discord interactions, and `review_post_requests` becomes the input to the Review Integrity Check. The points and accomplishments work does not depend on Discord at all, so it ships first, during the Skool-only phase, and your hand-kept Assist sheet becomes its seed data.

One naming collision needs a decision before anything else: the repo's marketplace is called "Proof Lab" (`/proof-lab` , `proof_lab_listings` , `proof_lab_deal_requests`), and so is your free Skool group. Members will hit both.

What fiveup has today

Next.js 16, React 19, Supabase (SSR client, RLS on every table, logic in `security definer` RPCs), Resend for email, deployed at `fiveup-nine.vercel.app`. 33 migrations, 25 tables, ~80 RPC functions, 79 commits, phases 5-16 documented in `docs/` . The app is far more complete than the July status file suggests; phases 8-16 added the public pages, moderation, visibility model and match preferences that file lists as missing.

Module	Tables	Key RPCs	UI	State
Identity and profiles	user_profiles, user_roles, plan_feature_gates, profile_external_links	handle_new_auth_user, update_my_profile, claim_public_username, update_publishing_settings, plan_feature_*	/account, /account/public, /signup, /login	Solid
Assets (the thing reviewed)	assets, asset_screenshots, asset_channels, asset_feedback_formats	create_asset, set_asset_visibility, set_asset_brand_visibility, set_asset_searchable	/assets/new, dashboard Assets tab	Solid
Web matching	matches, user_monthly_usage, match_blocked_channels	eligible_match_candidates, create_match, plan_total_match_limit, update_match_preferences	/browse, /account/preferences, dashboard Matches tab	Works; this is what Discord replaces
Member graph	member_match_edges, member_separation_cache, member_graph_state	member_shortest_path_degree, refresh_*_edges, pair_respects_separation_preferences	None (internal)	Solid; keep and feed from Discord
Feedback and stars	feedback_submissions, member_feedback_ratings, review_post_requests, public_feedback_permissions	submit_feedback, rate_member_feedback, request_review_post, approve_public_feedback, refresh_profile_feedback_rating	Feedback modal in dashboard, public feedback on /u/[username]	Solid; these are the internal stars
Proof Lab marketplace	proof_lab_listings, proof_lab_deal_requests, proof_lab_reviews, proof_lab_categories, charities	Full deal lifecycle, fundraiser leaderboard, notify-seller_email	/proof-lab, dashboard Proof Lab tab	Works; name collides with the Skool group
Moderation	blocked_phrases, moderation_flags, moderation_actions	report_content, record_auto_flag, resolve_flag, list_moderation_queue, outcome emails	/admin	Solid; the approval-queue pattern to copy
Public layer	visibility enums, public_username, searchable flags	get_public_profile, get_public_asset, list_searchable_*, OG images	/u/[username], /a/[slug], sitemap	Solid; where points will show
Demo mode	is_demo on profiles	user_is_demo, viewer_is_demo	/demo, seed script	Keep

Not present anywhere: the word Discord, the word Skool, any points or XP concept, any accomplishments concept, squads, tracks or missions. `matches.source` is the one hook already in place; today it distinguishes browse-initiated from system-initiated matches and can take a `discord` value without a schema change.

Keep, archive, repurpose, add

Nothing in the database gets dropped. "Archive" means the UI moves out of the routing tree and the RPC stays callable behind a flag; "Repurpose" means the same rows get written by a new caller.

Disposition	What	Why
Keep	Auth, profiles, public usernames, plan gates, external links, assets and screenshots, feedback submissions, member feedback ratings, public feedback permissions, moderation (all of it), public profile and asset pages, demo mode, Resend emails	This is the identity and reputation core. Discord cannot replace any of it
Keep, rename	Proof Lab marketplace (<code>/proof-lab</code> , listings, deals, charity pledges, seller reviews)	Real, working, and useful for Pro members selling services; but it cannot share a name with the Skool group. Working rename: "Deals" (<code>/deals</code>); tables keep their names
Archive	<code>/browse</code> (BrowsePage), <code>/account/preferences</code> (MatchPreferencesPage), the dashboard Matches tab's request and accept actions, <code>getBrowseQuota</code> , <code>getEligibleCandidates</code> as a member-facing call, monthly match quotas	The member-facing matching flow. Discord Squads and matches replace it; it comes back by flipping one flag
Repurpose	<code>matches</code> table and <code>create_match</code> RPC	A Squad pairing or a Discord match writes a <code>matches</code> row with <code>source = 'discord'</code> ; feedback, stars and the graph keep working unchanged

Disposition	What	Why
Repurpose	<code>member_match_edges</code> , <code>member_separation_cache</code> , <code>member_shortest_path_degree</code> , <code>pair_respects_separation_preferences</code>	The relationship graph, now fed by interactions of every type (assist, squad, collaboration, review), not only matches. Powers Squad recommendations and the Review Integrity Check
Repurpose	<code>eligible_match_candidates</code>	Becomes the Squad and match recommender the bot calls: same separation and preference logic, different consumer
Repurpose	<code>review_post_requests</code> and <code>match_blocked_channels</code>	The "please post this publicly on channel X" workflow becomes the Review Integrity Check form and its channel history
Repurpose	<code>list_moderation_queue</code> / <code>resolve_flag</code> pattern and the <code>/admin</code> page	Template for the points approval queue: same shape, same admin gate, same outcome email
Add	Discord identity, Skool identity, interactions, points ledger, accomplishments catalog and claims, assists, redemptions, squads and tracks (later)	The Discord-era data model, in the data model section
Add	Discord interactions endpoint under <code>src/app/api/discord/</code>	The bot, in the bot section
Add	Points, accomplishments and assists on <code>/u/[username]</code> and the dashboard	The public display Skool cannot give you

Target architecture

Supabase is the ledger, Discord is the room, ProofSignals is the record, Skool is the display. Discord never owns state: every `/assist`, `/claim` and Squad pairing becomes a row in Supabase within the same request, and if Discord vanished tomorrow the rows would still be there and the archived web flows would take over as the writer.

```

flowchart LR
    SK[Skool<br/>Proof Lab + Pro] -->|join, link| PS[ProofSignals<br/>Next.js  
on Vercel]
    DC[Discord<br/>Squads, Missions, /assist] -->|interactions endpoint|  
API[api/discord<br/>route handlers]
    API -->|service-role RPCs| DB[(Supabase<br/>ledger + graph)]
    PS -->|member RPCs| DB
    WEB[Archived web matching<br/>flag off] -.->|flag on| DB
    DB --> PUB[Public profile<br/>points, stars, assists]
    PUB -->|link back| SK
    ADM[Admin queue<br/>approve points] --> DB
  
```

The one switch: a `MATCH_SURFACE` setting (`discord` or `web`) read from `plan_feature_gates` so it lives with the other gates and can differ per plan. When it is `web`, `/browse` and preferences render and `create_match` accepts member calls. When it is `discord`, those routes redirect to the Discord invite and `create_match` accepts only service-role calls from the bot. Both paths write identical `matches` rows.

The bot runs inside this repo as Next.js route handlers, not as a separate always-on process. Discord slash commands can be served over HTTPS (the Interactions Endpoint), which Vercel handles fine, so there is no second deployment, no Railway box, and one set of environment variables. A gateway bot that listens to every message only becomes necessary if you want passive activity logging, and that is a Phase D decision, not a Phase A one.

Archive strategy: shelve without deleting

Archived code stays in the repo, compiles, and has a written restore path. Nothing archived lives only in git history, because git history is where things go to be forgotten.

1. Tag `main` as `pre-discord-refactor` before the first commit. Cut the work on a `discord-refactor` branch; merge in phases.

2. Move the shelved UI, not delete it: `src/components/fivestarz/BrowsePage.jsx` and `MatchPreferencesPage.jsx` go to `src/archive/web-matching/`, with the match request and accept pieces of `DashboardPage.jsx` extracted into `src/archive/web-matching/MatchActions.jsx`. The archive folder is imported only by the flag.
3. Routes stay: `/browse` and `/account/preferences` keep their `page.jsx`, which reads `MATCH_SURFACE` and either renders the archived component or redirects to the Discord invite. No 404s, no broken links in old posts.
4. RPCs stay, gated: `create_match` and `eligible_match_candidates` get a first-line check that raises unless the surface is `web` or the caller is the service role. One migration, reversible.
5. `data.js` keeps every function; the archived ones get a `// ARCHIVED: web matching surface`, see `docs/archive/web-matching.md` comment and no other change.
6. `docs/archive/web-matching.md` documents the restore: set the gate to `web`, nothing else. Test that restore once before merging Phase A so the claim is true.
7. The Proof Lab marketplace is not archived; it is renamed and stays live for Pro members.

What this buys you: the entire web matching product is one row update away from being back, the graph keeps accumulating relationship data the whole time Discord is the surface, and any member who ever used the old flow sees a redirect instead of a dead page.

Data model changes

Seven new tables, one enum, two new columns, and zero drops. Every table follows the repo's existing pattern: RLS on, writes only through `security definer` RPCs, `set_updated_at` trigger, admin override via `is_admin()`.

Table	Columns (key ones)	Purpose
<code>member_identities</code>	<code>user_id</code> , <code>provider</code> (<code>discord</code> <code>skool</code>), <code>external_id</code> , <code>external_handle</code> , <code>verified_at</code> , <code>linked_via</code>	One row per linked account. Discord via Supabase Auth's Discord provider and identity linking; Skool via email match on join (Zapier) plus a manual <code>/link</code> fallback

Table	Columns (key ones)	Purpose
interactions	id, kind (assist collaboration squad internal_review external_review_event match), member_a, member_b, context_type, context_id, evidence_url, occurred_at, source (discord web admin)	The universal event; everything in the reputation graph is one of these. Replaces one-table-per-behavior
accomplishments	id, slug, title, points, requires_evidence, requires_approval, active	The catalog: ship an app, ship a Skilllet, first paying client, Twilio 10DLC approved, and so on. Admin-editable, not hard-coded
accomplishment_claims	id, user_id, accomplishment_id, evidence_url, note, status (pending approved rejected), reviewed_by, reviewed_at	A member's claim, awaiting the queue
assists	id, helper_user_id, helped_user_id, interaction_id, what, evidence_url, status, reviewed_by	A gratitude nomination from the helped party. Fixed 1 point on approval, per your rule
points_ledger	id, user_id, delta, kind (accomplishment assist admin_adjust redemption), source_table, source_id, status, created_at	Append-only. Balance = sum of approved deltas; lifetime = sum of approved positive deltas. Never an updatable points column on the profile

Table	Columns (key ones)	Purpose
redemptions	id , user_id , kind (skilllet app_month pro_month pro_year), points_cost , status (requested approved fulfilled refunded), fulfillment_note	Spending, and the paid-group refund record

New columns: `user_profiles.points_balance_cached` and `lifetime_points_cached`, refreshed by a trigger on `points_ledger` the same way
`refresh_profile_feedback_rating` maintains the star averages today, so the public page never sums a ledger on read.

New enum value: `'discord'` on the existing free-text `matches.source` (it is text today, so this is a convention, not a migration).

Graph feed: a trigger on `interactions` calls the existing `refresh_member_match_edge_for_pair` and `bump_member_graph_version`, so Squads, assists and reviews all count toward separation degree exactly as matches do now. That is the whole "tie the graph to Discord" change.

Squads, tracks and missions get their own tables in Phase D, not now; until then a Squad is an `interactions` row of kind `squad` per pair, which is enough for the graph and the recommender.

Points economy in code

Every point enters through one RPC, `award_points`, which only the approval queue calls, and leaves through one RPC, `redeem_points`, which checks the balance inside the transaction. Members never write to the ledger; they create claims, assists and redemption requests, and an admin decision produces the ledger row.

Earning

Source	Who triggers	Approval	Points
Accomplishment (ship an app, ship a Skilllet, first paying client, Twilio 10DLC approved, and so on)	Member, via <code>/claim</code> in Discord or the dashboard, with an evidence URL	Admin queue	Per catalog row; suggested 25-100
Assist	The helped member, via <code>/assist @helper</code> or the dashboard	Admin queue (auto-approve later once patterns are known)	1, fixed
Admin adjustment	Admin only	None	Any, with a note
Internal stars, external reviews, Skool likes	Nobody	Never	0, enforced by having no code path

The queue is `/admin` with a second section beside the moderation queue:

`list_points_queue(status)` returns pending claims and assists with evidence links; `resolve_points_item(id, action, note)` approves or rejects, writes the ledger row, and sends the outcome email through the Phase 16 pattern. Same admin gate, same audit trail, same fail-open email.

Spending

`redeem_points(kind, points_cost)` inserts a negative ledger row and a `redemptions` row in one transaction, or raises if the balance is short. Fulfillment is manual for now: a Skilllet is a Lemon Squeezy 100%-off code sent by email, an app month is a plan change, a Pro month is the refund flow below. The catalog of what costs what lives in `plan_feature_gates` config so you can change prices without a deploy.

The paid-group refund mechanic

Skool bills the member directly and gives you no way to gift months, so the flow is: the member pays Skool as normal, then requests `pro_month` (or `pro_year`) as a redemption. You approve it in the queue, refund the corresponding amount to them inside Skool's

admin, and mark the redemption `refunded` with the Skool transaction reference in `fulfillment_note`. The ledger already debited the points at request time, so a rejected request gets a compensating positive row, never an edit.

Two guardrails: a redemption cannot take the balance below zero, and lifetime points (the public number) never decrease when points are spent. The profile shows both: lifetime as the badge, balance as the wallet, visible only to the member.

Skool tie-back

Skool gives you almost nothing to write into and only two things to read from, so the tie-back is a display strategy, not an integration. Official surface as of Sep 2026: the Zapier app needs a Skool Pro plan and offers two polling triggers (Answered Membership Questions, New Paid Member) and two actions (Invite Member, Unlock Course for Member), keyed by email ([Zapier help](#)). There is no official way to post, edit a member's points, or embed a live widget. An unofficial reverse-engineered API exists on Apify with ~33 actions ([ctala/skool-api-docs](#)); it works today and could stop working any day, so nothing load-bearing should depend on it.

Goal	Can Skool do it?	Workaround that ships
Show a member's lifetime points and stars inside Skool	No	The member's ProofSignals public page (<code>/u/[username]</code>) becomes the record; they paste it as the Website link on their Skool profile. A points and stars OG image (the repo already generates OG images) makes the link preview itself the badge
A leaderboard in Skool	Not natively	A weekly "Proof Board" post, generated by a route (<code>/api/skool/board</code>) that renders the top 10 by lifetime points, assists and stars as an image plus text you paste. Automate the paste via the unofficial API only if you accept it can break

Goal	Can Skool do it?	Workaround that ships
Know who joined Skool so their ProofSignals account links	Yes, partly	Zapier New Paid Member (Pro) posts to <code>/api/skool/member</code> and writes <code>member_identities(provider = skool)</code> by email match. Free-group joins have no trigger, so the free path is a <code>/link</code> step: member enters their Skool profile URL on <code>/account</code> , admin sees it in the queue
Unlock a Pro course as a points reward	Yes	Redemption approval fires Zapier's Unlock Course for Member; the first spend that needs no manual step
Gift Pro months	No	The refund mechanic in the points section, until Skool ships gifting
Replace Skool's like-based leaderboard	No	You cannot turn it off; you can make it irrelevant by pinning the Proof Board and naming Assist of the Week in every Friday post

The honest read: Skool is where people find you and read; every number that means something lives on proofsignals.net and gets linked into Skool, never the other way around. When Skool ships an API worth using, the tie-back is one route handler per capability, because everything is already in the ledger.

Discord bot

The bot is a set of Next.js route handlers in this repo that Discord calls over HTTPS. No gateway connection, no separate process, no second deploy. Discord signs every request with Ed25519; the handler verifies the signature, dispatches on the command name, calls a Supabase RPC with the service-role key, and answers within 3 seconds (or defers and follows up for anything slower).

Layout

```
src/app/api/discord/interactions/route.js    verify signature, dispatch
src/lib/discord/verify.js                    Ed25519 check (tweetnacl or
discord-interactions)
src/lib/discord/commands/                    one file per slash command
src/lib/discord/register.mjs                 registers commands with Discord
(run once per change)
src/lib/discord/identity.js                  discord_user_id -> user_id via
member_identities
supabase/migrations/2026xxxx_discord_rpc.sql
```

Identity linking

Add Discord as a Supabase Auth provider and use identity linking, so a member on `/account` clicks Link Discord and Supabase stores the Discord user id; the handler writes `member_identities`. Fallback for people who never open the site: `/link` in Discord returns a one-time code, they paste it on `/account`. Every command first resolves the Discord id to a `user_id` and refuses politely if unlinked, so unlinked members cannot earn or spend.

Commands at launch

Command	Writes	Notes
<code>/link</code>	<code>member_identities</code>	One-time code flow
<code>/assist @member what</code>	<code>interactions(kind=assist), assists(status=pending)</code>	Only the helped party can run it; helper cannot nominate themselves
<code>/claim <accomplishment> <evidence-url></code>	<code>accomplishment_claims(pending)</code>	Autocomplete from the <code>accomplishments</code> catalog
<code>/points</code>	none	Ephemeral reply: balance, lifetime, pending items
<code>/board</code>	none	Top 10 lifetime, assists, stars; same query as the Skool Proof Board

Command	Writes	Notes
<code>/rate @member</code>	none	Replies with the member's ProofSignals rating link; stars are given on the site, never in Discord, so the review flow stays first-party
<code>/squad</code> (Phase D)	<code>interactions(kind=squad)</code> per pair, later <code>squads</code>	Admin-only until the recommender is trusted

Every command posts a public embed in `#assist-log` or the Squad channel and an ephemeral confirmation, so nominations are visible to the room, which is the social pressure that keeps them honest.

Approval loop

Pending items appear in the `/admin` points queue. Approving sends the outcome email and, if the member is linked, a Discord DM through the bot's REST token. Rejections say why, in one line, using the same copy table pattern as moderation emails.

Later, if needed

A gateway listener (`discord.js`, on a small always-on host) only when you want passive signals: messages per Squad channel per week, thread participation, reaction counts. Those feed activity metrics, never points, and they are what the Phase 2 kill rule in Launch Strategy v2 measures.

Phased build plan

Six phases, each one a mergeable branch with its own migration, its own `verify-phaseNN.mjs` in the repo's existing style, and a rollback that is a gate flip or a migration down, never a revert of member data. Phases A and B ship during the Skool-only period and need no Discord server; C through F line up with gates G2 and G3 in Launch Strategy v2.

Phase	Scope	Done when	Rollback
A · Shelve and switch (1 week)	Tag <code>pre-discord-refactor</code> ; <code>MATCH_SURFACE</code> gate; move web matching to <code>src/archive/</code> ; routes redirect; gated RPCs; rename marketplace routes to <code>/deals</code> ; <code>docs/archive/web-matching.md</code>	Flipping the gate to web restores Browse end to end on a preview deploy	Set gate to <code>web</code>
B · Points without Discord (2 weeks)	Migration: <code>interactions</code> , <code>accomplishments</code> , <code>accomplishment_claims</code> , <code>assists</code> , <code>points_ledger</code> , <code>redemptions</code> , cached columns, graph trigger; RPCs <code>award_points</code> , <code>redeem_points</code> , <code>list_points_queue</code> , <code>resolve_points_item</code> ; admin queue UI; claim and assist forms on the dashboard; points, assists and accomplishments on <code>/u/[username]</code> and its OG image; import of the hand-kept Assist sheet	You approve a real claim from the queue and it shows on a public profile	Hide the profile section behind a gate; tables stay
C · Discord commands (1-2 weeks, before G2)	<code>member_identities</code> ; Discord auth provider and identity linking; interactions endpoint; <code>/link</code> , <code>/assist</code> , <code>/claim</code> , <code>/points</code> , <code>/board</code> , <code>/rate</code> ; DM on approval	Five test members run every command in the private server and the rows are correct	Unregister commands; endpoint returns 503; no data change
D · Graph and Squads (2 weeks, after G2)	<code>squads</code> , <code>tracks</code> , <code>squad_members</code> ; <code>eligible_match_candidates</code> repurposed as <code>recommend_squad</code> ; <code>/squad</code> admin command; Squad pairings write <code>matches(source=discord)</code> so feedback and stars flow; Skool Proof Board route and Zapier member webhook	First Squad is formed from a recommendation and one member rates another through the existing feedback flow	Squads become manual; recommender unused

Phase	Scope	Done when	Rollback
E · Redemptions (1 week)	Spend flow on <code>/account</code> ; Pro-month refund workflow with <code>fulfillment_note</code> ; Zapier Unlock Course action on approval; price catalog in <code>plan_feature_gates</code>	One member spends points on a Skilllet code and one on a Pro month, both fulfilled	Disable spend; ledger untouched
F · Review Integrity Check (1 week, toward G4)	Reuse <code>review_post_requests</code> as the private form; <code>check_review_integrity(reviewer, subject, channel)</code> reads separation degree, prior reviews, Squad and assist history; verdict shown to admin and member	A high-overlap pair gets the internal-feedback recommendation instead of a green light	Form only; no verdict logic

Sequencing rule: B ships before C so the first Discord command has a working queue behind it, and D waits for G2 because a recommender with 40 members recommends the same four people forever.

Estimate for the whole thing at your pace with Claude Code: 8–9 working weeks spread over October through January, which lands F right around the G4 target. Phases A and B are the only ones on the critical path for the next 30 days.

Open questions

- ☐ Proof Lab name collision: rename the marketplace to Deals (my pick), rename the Skool group, or keep both and live with it?
- ☐ Does the marketplace stay live for members during the refactor, or go dark until Pro opens? It works today; I lean live, Pro-only.
- ☐ Plan codes are `sprout` / `bloom` / `flourish` . Map them to Proof Lab free / Proof Lab Pro, or add a new code? This decides which gates Pro members get.
- ☐ Point values for the first accomplishments catalog: ship an app, ship a Skilllet, first paying client, Twilio 10DLC approved. Give me numbers or let me propose a table.
- ☐ Assist points: fixed 1 as you said, with breadth weighting (same pair pays less over time) or flat forever?
- ☐ Redemption prices: how many points for a Skilllet, an app month, a Pro month, a Pro year?

- ☐ Skool plan: are you on Hobby or Pro? Zapier (member webhook, course unlock) needs Pro.
- ☐ Who approves the queue: only you, or do moderators (`is_moderator`) get it too?
- ☐ Discord identity: Supabase Discord auth provider with identity linking (needs a Discord application and OAuth redirect), or the `/link` code flow only?
- ☐ Access: the repo is public so I read it fine. For the build, do you want me working in Claude Code against a branch, or drafting each phase's migration and files here for you to paste?
- ☐ Email domain: outcome emails go out from `noreply@bendersaas.ai` today. Keep, or move to an indieops.co or proofsignals.net sender before members see them?